

# CASCADE: Constraint-Aware Scaffolded Decomposition for Logical Anomaly Detection

Jinchan Kim  
Ajou University  
Suwon, Republic of Korea  
jinchank0959@ajou.ac.kr

Jinhee Im  
Ajou University  
Suwon, Republic of Korea  
jinhee8553@ajou.ac.kr

Hyunsouk Cho  
Ajou University  
Suwon, Republic of Korea  
hyunsouk@ajou.ac.kr

**Abstract**—Logical Anomaly Detection (LAD) requires verifying whether all category-specific constraints, such as object counts, pairing relations, and measurement ranges, are satisfied by a query image given only a few normal references. Recent LAD methods based on Large Vision-Language Models (LVLMs) formulate this as checklist verification, decomposing the decision into independent textual checks. **This decomposition, however, is text-only. Every item still reads the same unmodified query image, so the LVLM must extract different forms of visual evidence from one raw image. We identify this text-image mismatch as a constraint-specific perception gap, the source of two failure modes: instance conflation for binding constraints and scale drift for calibration constraints.** To diagnose whether these failures stem from insufficient reasoning or insufficient perception, we compare textual reasoning guides and visual perception guides in a training-free setting. Our analysis shows that reasoning over the same image provides little benefit, whereas adapting the visual input to each constraint substantially **improves** verification accuracy. Based on this finding, we propose CASCADE, a training-free framework that extends checklist decomposition using constraint-specific visual scaffolds. We further introduce CASCADE++ which adapts these scaffolds at the object level by deriving their configurations from the constraint structure and reference distribution. **Across MVTec-LOCO and VID-AD, CASCADE++ surpasses the full-shot state of the art by 3.9 AUROC points with only three references, and improves over the strongest VID-AD baseline by 11.4, transferring across multiple LVLM backbones without any training. Our code is available at <https://github.com/edjjincode/CASCADE>.**

**Index Terms**—Logical Anomaly Detection, Large Vision-Language Models, Visual Scaffolding, Training-Free Anomaly Detection, Constraint Verification

## I. INTRODUCTION

Driven by recent advances in vision-language models, Logical Anomaly Detection (LAD) [1] has emerged as an important task in manufacturing inspection. Traditional anomaly detection [2], [3] focuses on identifying pixel-level defects such as cracks, scratches, and contamination. LAD, by contrast, requires models to verify logical constraints, such as object counts, pairing relationships, or measurement ranges, from only a few normal references and a query image. Since violating even a single constraint renders the instance anomalous, every constraint must be verified without exception.

To verify the validity of a query image with only a few normal references, recent approaches have adopted Large Vision-Language Models (LVLMs) by formulating the task as a checklist evaluation. These LVLM-based LAD methods [4],

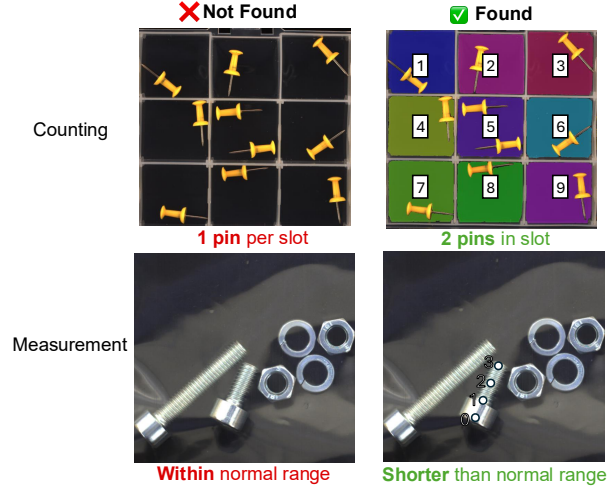


Fig. 1: Comparison of checklist verification without versus with visual perceptual guides. (Left) LVLMs struggle with accurate counting and measurement on raw images. (Right) Applying constraint-specific guides, such as coloring objects and coordinate grids, makes the relevant instances and measurements explicit, enabling the model to successfully detect the anomalies.

[5] decompose the overall verification into independent per-constraint checks by explicitly converting constraints into a checklist. First, in the generation phase, each logical constraint is converted into a textual checklist item that an LVLM can evaluate. Then, in the verification phase, each checklist item is verified against the query image individually. This checklist-based formulation is useful because it reduces the chance of overlooking a constraint violation and makes it possible to trace which constraint was violated.

**This decomposition, however, creates a mismatch between text and image. The checklist tailors the question to each constraint, yet every check still reads the same unmodified raw query image. The LVLM must therefore extract different types of visual evidence from an identical input, despite the fact that different constraints demand fundamentally different perceptual operations.** Count and pairing constraints require the model to individuate and bind visually similar instances, whereas length, ratio, and fill-level constraints require the model to

compare continuous measurements against reference-defined normal ranges. This mismatch manifests as two recurring failure modes in LAD: instance conflation, where visually similar objects are not reliably separated, and scale drift, where measurement judgments become inconsistent across references and queries. To determine whether these failures stem from insufficient reasoning or insufficient perception, a direct diagnosis is needed.

To diagnose this under the training-free setting, we compare two input-level interventions that require no model update. The first is a textual reasoning guide [6], [7] that keeps the image unchanged but prompts more deliberate reasoning over each checklist item. The second is a visual perceptual guide [8]–[10] that modifies the image itself by making constraint-relevant visual information explicit. [A textual guide that resolves the errors would locate the limitation in reasoning; a visual guide that does so would locate it in perception.](#)

Our analysis confirms that the limitation is perceptual, and that the visual input must be adapted per constraint alongside the checklist. As illustrated in Figure 1, providing visual perceptual guides makes the required visual instances and measurement bases explicit, enabling the model to successfully detect anomalies. The scaffold must be constraint-specific, as different constraints demand different forms of perception. Binding constraints require separating and associating visually similar instances, which mark-based scaffolds [11] make explicit, whereas calibration constraints require comparing measurements against reference-defined normal ranges, for which coordinate-based scaffolds [9] supply a frame.

Based on this principle, we propose **CASCADE**, a training-free framework that [selects the scaffold type from the constraint and overlays it on the query image, so that each check reads a view adapted to what it examines.](#) **CASCADE** alone, however, leaves the scaffold’s configuration at a fixed default, and a default that misaligns with the constraint can mislead the verifier more than no scaffold at all. We therefore extend it to **CASCADE++**, which derives each scaffold’s configuration from the constraint’s logical structure and the reference distribution. Our contributions are summarized as follows.

- We diagnose the limitation of checklist-based LVLM verification by comparing textual reasoning guides and visual perceptual guides, showing that LAD failures [stem from insufficient constraint-relevant visual perception rather than from insufficient reasoning.](#)
- We propose **CASCADE**, a training-free framework that routes each constraint to its matched scaffold type, and extend it to **CASCADE++**, which derives the scaffold’s configuration from the constraint’s logical structure and the reference distribution.
- With three references and no training, **CASCADE++** reaches 93.2 AUROC on the MVTec-LOCO logical subset, exceeding the full-shot state of the art by 3.9, and 94.5 on VID-AD without any parameter adjustment, exceeding the strongest baseline by 11.4. Our code, full prompt templates, constraint parser, calibration probes, and grid solver are publicly available.

## II. RELATED WORK

### A. Vision-based LAD

Vision-based methods detect anomalies as deviations from learned normal patterns. Full-shot methods such as PatchCore [2] and EfficientAD [3] achieve strong structural anomaly detection but degrade on logical anomalies. CSAD [12] and SALAD [13] address this gap by modeling component structure, yet they still require large training sets [per category](#). Training-free methods such as SAM-LAD [14] and UniVAD [15] [alleviate this requirement](#) by leveraging foundation models. Despite these advances, all vision-based methods share a fundamental limitation: [they](#) output only a scalar or matching score, and cannot identify which constraint was violated or explain why.

### B. LVLM-based LAD

Recent methods shift from deviation measurement to direct constraint verification by feeding natural-language constraints to LVLMs. LogicAD [5] and LogicQA [4] convert constraints into textual checklist items, [LogiCode \[16\] compiles them into executable programs](#), and LogSAD [17] [routes the task through multiple vision-language foundation models, together enabling traceable constraint-level verification.](#) [Across these approaches, however, the decomposition operates on the text side alone.](#) The check is tailored to the constraint, but every check still reads the same unmodified query image. Relying entirely on the LVLM’s internal perception, these methods add no visual signal to [compensate for the perceptual failures that verification in this setting exposes.](#)

### C. Visual Scaffold for LVLMs

Visual scaffolds compensate for perceptual limitations of LVLMs by adding training-free visual information onto images and have proven effective across a range of vision tasks [11], [18]. However, existing scaffolds derive their parameters from image space and do not reflect the constraint structure that defines logical anomalies. *Disambiguation scaffolds* such as SoM [11] make duplicate instances visually distinguishable, yet their ID numbering follows the spatial order of the image and is unrelated to the binding structure of the constraint. *Alignment scaffolds* such as SCAFFOLD [9] and VISER [10] provide a cross-image reference frame for continuous measurements, yet their spacing and offset are fixed and independent of the normal range defined by few-shot references; the same holds for [Bbox+coords](#), the [box-and-coordinate](#) form we adapt as a baseline from the drawing tools of Visual Sketchpad [18]. Visual Sketchpad itself is an *agentic scaffold*, dynamically selecting tools such as object detection, segmentation, and zoom-in/cropping based on query-time perception. In LAD this becomes self-defeating, since the scaffold is needed precisely because that perception is unreliable. **CASCADE++** breaks this dependency by deriving [the scaffold](#) configuration from [the constraint text and the reference distribution](#), so that the scaffold directly reflects the logical structure of the constraint.

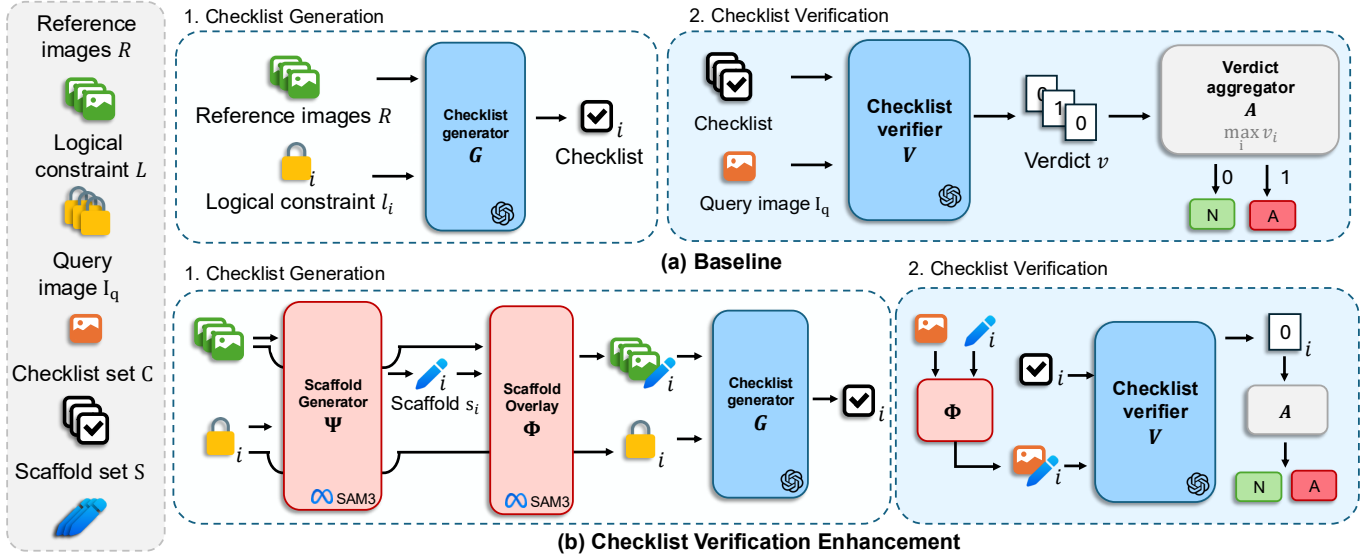


Fig. 2: Method comparison between the baseline and CASCADE. Both methods use checklist generator and checklist verifier for verdicts. Unlike the baseline, ours overlays a constraint-specific scaffold on the query image, so each constraint is verified with an adapted visual input.

### III. PRELIMINARIES

Logical anomalies are defined as violations of category-specific constraints [1]. Attributing an anomaly to the constraint it violates requires those constraints to be explicit inputs to the detection system. A reference image is a normal sample that defines what a correct instance of the category looks like. We write  $R = \{I_{r_1}, \dots, I_{r_K}\}$  for the set of  $K$  reference images, and we set  $K = 3$  throughout our experiments. A query image  $I_q$  is the test-time input whose normality must be decided.

Not all constraints are verified in the same way, and this difference in the underlying perceptual operation motivates our approach. Each category is governed by a set of logical constraints  $L = \{l_1, \dots, l_M\}$ , where  $M$  is the total number of constraints, and each constraint  $l_i$  states a normal condition on one property of the category. We group them into two kinds according to the operation their verification demands. A *binding* constraint concerns discrete structure and is checked by counting or matching, such as requiring exactly *one pushpin* per compartment or a correct pairing between groups. A *calibration* constraint concerns a continuous measurement and is checked by comparison against the references, such as a cable length matching the reference instances *or a liquid fill level falling within the reference range*. This binding–calibration split organizes our analysis in Section V.

Given a query image  $I_q$ , the LAD task decides whether it satisfies all constraints in  $L$ . Since logical anomaly detection is defined over multiple constraints, the decision is made through per-constraint verification. For each  $l_i$ , the system returns a binary verdict  $v_i \in \{0, 1\}$ , where  $v_i = 0$  means satisfied and  $v_i = 1$  means violated. The query image is normal only if all verdicts are zero.

### IV. FRAMEWORK

Existing LVLM-based methods verify each constraint independently through a checklist-based framework [4], [5], [16], [17], organized in three stages: checklist generation prepares each constraint into a form an LVLM can evaluate from the constraint text and the references; checklist verification evaluates each prepared item against the query to produce a verdict; and verdict aggregation combines the per-constraint verdicts into an image-level decision, anomalous if even one constraint is violated.

#### A. Baseline: LVLM-based Framework

*Checklist Generation:* A generator  $G$  converts each constraint  $l_i$  into an independently evaluable item  $c_i$  using the reference set  $R$ , yielding the checklist  $C = \{c_1, \dots, c_M\}$  via  $c_i = G(l_i, R)$  for  $i = 1, \dots, M$ . Existing methods differ only in the output format—a structured JSON record of target object, property, and normal condition in LogicAD [5], a natural-language yes-or-no question in LogicQA [4]—and in both, each item is converted independently and is self-contained for verification.

*Checklist Verification:* A verification operator  $V$  evaluates each item against the query image in a single LVLM call and returns a binary verdict  $v_i$ .

$$v_i = V(c_i, I_q), \quad i = 1, \dots, M \quad (1)$$

The text input  $c_i$  varies per constraint, reflecting the per-constraint decomposition. The image input  $I_q$ , however, is the same raw query image for every constraint. Whether a constraint examines the count of identical objects or the length of a component, the call receives the identical, unmodified

image. The question is tailored to the constraint, but the visual evidence is not.

**Verdict Aggregation:** Because a product is defective if it violates even a single constraint, the query image is classified as anomalous whenever at least one constraint is violated. Given binary constraint-level verdicts  $\{v_i\}_{i=1}^M$ , where  $v_i = 1$  indicates a violated constraint, we define the image-level abnormal verdict as  $A(I_q) = \max_{1 \leq i \leq M} v_i$ , where  $A(I_q) = 1$  denotes an abnormal image and  $A(I_q) = 0$  denotes a normal image. AUROC evaluation requires a continuous score, for which we use the violated fraction  $\text{Score}(I_q) = \frac{1}{M} \sum_{i=1}^M v_i$ . Since  $\text{Score}(I_q) > 0$  exactly when at least one constraint is violated, the score is positive precisely on the images  $A(I_q)$  marks anomalous.

### B. Checklist Verification Enhanced Framework

The decomposition introduced by the checklist applies to the text alone. Checklist verification enhancement extends it to the image: each verification call reads a scaffolded query in place of the raw query, with no model parameters updated.

This enhancement adds two operators to the baseline pipeline while leaving generation and verification otherwise intact. A scaffold-generation operator  $\Psi$  reads a constraint and the references and returns a scaffold configuration, and a scaffold-rendering operator  $\Phi$  overlays that configuration onto an image. Given  $l_i$  and  $R$ ,  $\Psi$  produces the configuration  $s_i = \Psi(l_i, R)$ , and  $\Phi$  renders it onto each reference as  $\hat{I}_{r_k}^i = \Phi(I_{r_k}, s_i)$ , forming the scaffolded reference set  $\hat{R}^i = \{\hat{I}_{r_1}^i, \dots, \hat{I}_{r_K}^i\}$ .

The checklist item is then generated from the constraint and the scaffolded references as  $c_i = G(l_i, \hat{R}^i)$  for  $i = 1, \dots, M$ , so that the question matches what the scaffold exposes. This mirrors the baseline checklist generation, but replaces the raw reference set  $R$  with the scaffolded reference set  $\hat{R}^i$ .

At verification time,  $\Phi$  overlays the same configuration onto the query, producing a scaffolded query  $\hat{I}_q^i$  tailored to constraint  $l_i$ , which the verifier reads in place of the raw query.

$$\hat{I}_q^i = \Phi(I_q, s_i), \quad v_i = V(c_i, \hat{I}_q^i), \quad i = 1, \dots, M \quad (2)$$

Whereas the baseline tailors only the text per constraint, this enhancement tailors both the text and the image, so that the per-constraint decomposition applies to the visual evidence as well. Verdict aggregation is unchanged from the baseline. The scaffold form follows the constraint type: a disambiguation scaffold for a binding constraint and an alignment scaffold for a calibration constraint. Section V establishes which form each type requires, and Sections VI and VII develop how  $\Psi$  configures it.

## V. REASONING AND PERCEPTION GUIDES FOR THE VERIFICATION PHASE

In this section, we analyze the two guides available for the verification phase: a textual reasoning guide that reasons more carefully over the checklist item, and a visual guide that modifies the query image the verifier reads. We empirically test the available forms of each guide across binding and calibration constraints, establishing which overlay each constraint type

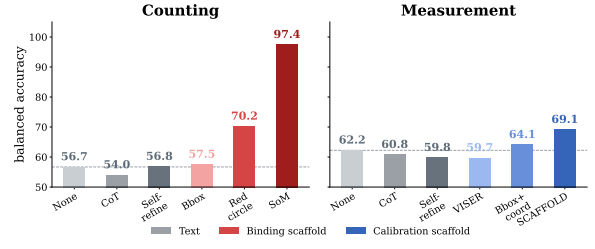


Fig. 3: Comparison of textual reasoning and visual perception guides. Bars show balanced accuracy on a binding constraint for Counting and the average over calibration constraints for Measurement, with the dashed line indicating the text-only baseline. The first three bars in each panel are the shared textual guides; the last three are the overlays applicable to that constraint type, which necessarily differ, since an identity overlay does not apply to a continuous measurement nor a spatial frame to counting.

requires. These findings fix the scaffolds that **CASCADE** implements.

We adopt GPT-5-image-mini [19] as the single backbone and report per-constraint Balanced Accuracy [20],  $BA = (\text{TPR} + \text{TNR})/2$ , since each constraint yields a single binary verdict, for which AUROC is undefined; we therefore report BA at the constraint level and AUROC only at the image level. The probes in Section V-B are synthetic images that vary a single factor in isolation. The comparisons below use a subset of MVtec-LOCO’s 16 logical constraints, chosen to isolate each failure mode most sharply; the full-system evaluation on all of them appears in Section VIII-B.

### A. Effectiveness of Textual Reasoning Guide

We first test whether a textual reasoning guide can aid the checklist verification. Since the checklist item  $c_i$  is the text input to the verifier, a natural means of improving verification is to have the model reason more deliberately over  $c_i$  before committing to a verdict. Such reasoning may be strengthened along two complementary directions, reasoning more thoroughly toward the verdict and revising it post hoc. We evaluate one guide for each, Chain-of-Thought [6], which prompts the model to reason step by step before answering, and Self-Refinement [7], which allows it to revise its own verdict. Both replace the item  $c_i$  with a reasoning-guided item  $c'_i$  while the query image  $I_q$  stays fixed, so the verdict becomes  $v_i = V(c'_i, I_q)$ .

Strengthening reasoning along either direction, however, yields no measurable gain. As Fig. 3 shows, neither guide improves on the text-only baseline by more than a point on binding and calibration constraints alike. Reasoning more deliberately over the same text therefore does not restore the verdict. If the limitation does not lie in the reasoning, it must lie in what the model perceives from the image, which we examine next.

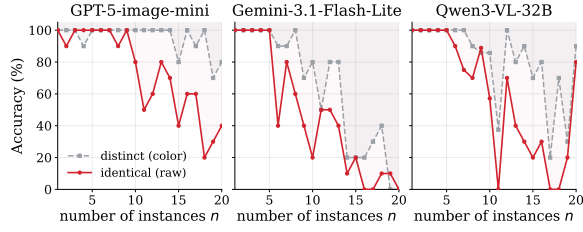


Fig. 4: Counting probe. We query the count of  $n$  identical circles on a blank canvas, varying only  $n$ . Raw-image accuracy (red) collapses past  $n \approx 10$ , while distinctly coloring each circle (gray) keeps it far higher throughout. The gap is consistent across three backbones, so the limitation is structural rather than backbone-specific.

### B. Effectiveness of Visual Guide

We next test whether a visual guide can recover what the textual guide could not. A visual guide overlays a training-free cue onto the query image, producing a guided query  $\hat{I}_q^i$  while leaving the checklist item  $c_i$  unchanged, so the verdict becomes  $v_i = V(c_i, \hat{I}_q^i)$ . This renders the property a constraint examines directly observable to the verifier while keeping the backbone and the question fixed. We evaluate the two overlay types reported in prior work, a disambiguation overlay that assigns each instance a separate visual identity and an alignment overlay that imposes a spatial reference frame for continuous measurements. A single disambiguation overlay raises binding Count from 56.7 to 97.4 (Fig. 3), a margin no textual guide approached. Since only the image changed while the backbone and question remained fixed, the limitation is perceptual rather than one of reasoning.

No single overlay, however, serves both: an identity overlay provides no reference frame for a continuous measurement, and a spatial frame imposes no identity on visually identical objects. The two types therefore fail for distinct reasons, which we examine in turn.

1) *Instance Conflation on Binding Constraints:* We begin with the first perceptual guide type, a disambiguation guide for binding constraints, where the verifier must individuate instances that look alike. We ask whether providing a perceptual guide that makes each instance visually distinct can restore this ability. Without such a guide, the model registers shared attributes such as shape and color yet fails to bind them into distinct instances [10], [21]. If it could individuate them, count accuracy would remain stable as their number grows. A synthetic probe that counts  $n$  identical circles shows the opposite: raw-image accuracy collapses past  $n \approx 10$  while per-circle coloring keeps it stable, consistently across three backbones (Fig. 4) [19], [22], [23]. Since color is the only factor altered, instance conflation is structural rather than an artifact a larger backbone would resolve.

A per-instance color restores this separation, yet separation alone does not determine the outcome. Among overlays that all separate the instances, accuracy still diverges widely, from 70.2 for a red circle to 97.4 for SoM (Fig. 3). Separation is therefore necessary but not sufficient, and what distinguishes

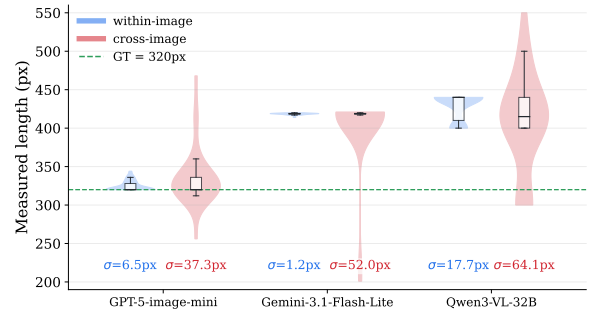


Fig. 5: Scale probe. We render a bar of fixed true length with a ground truth of 320 px and read its length from the LVLm. A within-image reading repeats the same image, while a cross-image reading spans separate renderings of the same length. Within-image readings stay tight, whereas cross-image readings spread several-fold to more than fortyfold across backbones, widening most on the most precise reader.

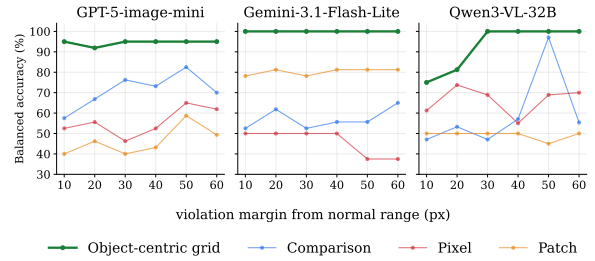


Fig. 6: **Range-membership probe.** Balanced Accuracy across violation margins for three VLM backbones. A test bar is judged against the range of  $K=3$  references, calibrated in code as in **CASCADE**, so the backbone reads only the test: Comparison sees the references too and answers directly, Pixel reports raw pixels, Patch units of an imagined  $12 \times 12$  grid, and the object-centric readout an index off a scale drawn on the bar; the last three are range-checked in code. The object-centric readout stays near-perfect at the smallest margins on two backbones and degrades last on the third, whereas the others drift and change ranking across models.

these forms is examined in Section VII. What the probe establishes is the overlay type: a per-instance color is what a binding constraint requires, and **CASCADE** renders it over **SAM3** masks (Section VI).

2) *Scale Drift on Calibration Constraints:* We turn to the second perceptual guide type, an alignment guide for calibration constraints, where the verifier must read the same physical length consistently across the references and the query. The model struggles to do this because it holds no absolute scale and re-estimates the measurement on each image. If it held a consistent scale, the same length would read identically on whichever image carried it. Instead, reading a fixed-length bar across separate renderings spreads 3.6 to 44 times wider than reading it within one repeated image (Fig. 5), so the drift is structural rather than reading noise. The readings are also biased upward relative to the ground truth, but a constant bias is absorbed by a reference-relative grid, since the references

and the query are read on the same scale; what such a grid cannot absorb is the cross-image spread. This drift is decisive near the normal-range boundary, where only a grid sustains accuracy as the violation margin shrinks (Fig. 6).

Removing the drift requires a grid that is discrete, two-dimensional, and object-anchored. A continuous coordinate would reinherit the drift, a single axis could not span a two-directional measurement, and a frame-fixed grid would shift cells as the object moves. The overlays that violate these requirements break down accordingly, as the single-axis VISER collapses on Length and the image-frame Bbox+coords collapses on Ratio, whereas the object-anchored grid holds across every calibration constraint. An object-anchored two-dimensional grid is thus the overlay a calibration constraint requires.

The two probes identify the overlay each constraint type needs, a per-instance color for binding constraints and an object-anchored grid for calibration constraints. **CASCADE** implements these two overlays (Section VI). The overlay alone, however, does not settle the verdict. A color separates the instances but imposes no order on how they relate, and a grid frames the measurement but not at a resolution matched to any particular constraint. How each overlay must then be configured, from the constraint and from the references, is what **CASCADE++** resolves (Section VII).

## VI. CASCADE: CONSTRAINT-AWARE SCAFFOLDED DECOMPOSITION

**CASCADE** implements the two overlays the analysis identified, routing each constraint to the one its type requires. Where the checklist decomposes the question per constraint, **CASCADE** decomposes the image per constraint, leaving all model parameters unchanged.

### A. Pipeline

**CASCADE** runs in four stages (Fig. 2 (b)). First, a parser reads the constraint  $l_i$  and returns its type  $\tau \in \{\text{binding, calibration}\}$  together with the target objects  $T$  the constraint acts on. Second,  $\Psi$  generates a type-matched scaffold configuration  $s_i$ , a disambiguation scaffold for a binding constraint and an alignment scaffold for a calibration constraint. Third, the checklist item  $c_i$  is generated from the scaffolded references. Fourth,  $\Phi$  overlays  $s_i$  on the query, the item is verified in a single LVLM call, and a rule check returns the binary verdict  $v_i$ , aggregated as in the baseline.

Both scaffolds require grounding the target objects  $T$  as masks. We adopt SAM3 [24], a pretrained foundation segmentation model used without any task-specific training, following the common practice of rendering scaffolds on segmentation masks rather than raw pixels [11]. One segmentation pass per image serves all constraints, each using only the masks its own targets name.

### B. Disambiguation Scaffold for Binding Constraints

A binding constraint requires telling identical instances apart. The disambiguation scaffold provides this by assigning a per-instance visual identity. Each target set is segmented into one

mask per instance with SAM3, a distinct color is overlaid on each mask to delineate its extent, and a numeric ID is written on top. The color separates the instances, and the ID lets the verifier refer to them when a constraint relates rather than merely counts them. In this stage  $\Psi$  assigns the IDs sequentially in image order; deriving them from the constraint is the subject of Section VII-A.

### C. Alignment Scaffold for Calibration Constraints

A calibration constraint requires reading a continuous measurement on a scale that is consistent across images, which the analysis traced to scale drift and showed is removed by an object-anchored two-dimensional grid (Section V-B2). **CASCADE** recovers a rotated bounding box from the SAM3 mask, aligned to the object’s principal axes, and draws the grid on this box so that the same physical endpoint maps to the same cell across the references and the query. The verdict is read as a comparison of cells: the references occupy a set of cells defining the normal range, and the query is judged normal when its endpoint falls in that range and a violation otherwise. In this stage  $\Psi$  returns a single fixed cell width; calibrating it to the references is the subject of Section VII-B.

## VII. CASCADE++

**CASCADE++** replaces the two fixed defaults that  $\Psi$  returns in **CASCADE** with configurations derived from the constraint and the references. In each case it is the derivation, not the overlay form, that determines the verdict.

### A. Constraint-Derived Numbering

A disambiguation overlay must lay out the very reasoning the constraint demands, so its numbering has to follow the constraint’s logical structure rather than the image. The model does not merely read the IDs off the overlay but follows them as the order in which to relate the instances, so the numbering becomes the path its reasoning takes. Color separates the instances and raises *Count* to 79.2, but, lacking such a path, recovers only part of *Linking* (61.4  $\rightarrow$  74.3). A numeric ID supplies one, yet where the path leads is what decides the verdict. The sequential ID of SoM runs along image position rather than the constraint, and the model, following it faithfully, is led toward the wrong pairing: *Linking* falls to 66.7, below color alone, and an offset ID drops to 38.6, below using no overlay at all (Table I), since a path that actively misleads is worse than none to begin with.

$\Psi$  therefore numbers the instances from the constraint rather than the query image. The parser identifies the instance sets the constraint acts on, for example the pushpins in the left compartment and those in the right, and the LVLM decomposes the constraint into the order within each set and the pairing across sets. Each SAM3 mask receives its ID in this order. SoM and **CASCADE++** share an identical visual form and differ only in this numbering rule, yet aligning the path with the constraint raises *Linking* by over twenty points, from 66.7 to 88.3 (Table I). What determines the verdict is not the overlay’s form but whether the path it lays out aligns with the constraint.

TABLE I: Disambiguation channel decomposition for binding constraints on Balanced Accuracy. We vary the two components of the overlay, color and a numeric ID, together with the ID’s numbering scheme, holding the question fixed so that only the overlay changes. Count is the Pushpins (PP) count constraint, and Linking the Splicing-Connector (SC) slot-matching constraint.

| Channel                       | Numbering                 | Count (PP)  | Linking (SC) |
|-------------------------------|---------------------------|-------------|--------------|
| None                          | —                         | 52.8        | 61.4         |
| Color only                    | —                         | 79.2        | 74.3         |
| ID only                       | offset                    | 75.8        | 38.6         |
| ID only                       | constraint-derived        | 79.3        | 86.2         |
| SoM (color + ID)              | sequential                | 97.4        | 66.7         |
| Visual Sketchpad              | self-scaffolding          | 49.8        | 69.5         |
| <b>CASCADE++ (color + ID)</b> | <b>constraint-derived</b> | <b>99.6</b> | <b>88.3</b>  |

### B. Reference-Calibrated Grid

An alignment grid must place its boundaries where the constraint’s normal range actually ends. Its cell width and position therefore have to follow the reference distribution rather than a fixed default. The model judges a query normal when its endpoint falls in the same cell as the references and a violation otherwise, so the cell width sets the resolution of that judgment. A cell narrower than the references scatter splits them across cells and flags a normal query, while a cell wider lets a violation fall in the reference cell and pass. The cell must therefore hold every reference reading and no more.

No fixed width meets this across constraints, because the references scatter by different amounts depending on what the constraint measures. A two-way ANOVA over cell widths and constraints (six widths  $\times$  six constraints, five reference sets each) confirms this, with the constraint-by-width interaction accounting for the largest share of variance ( $\eta^2 = 0.449$ ,  $p < .001$ ). Fitting the width to the object’s size fails for the same reason, since how far the readings scatter is unrelated to object size, collapsing to 25.6 on JB-Liquid and to chance on SB-Length (Table II). The position matters equally, as a boundary falling between the references splits the normal range even at the correct width.

$\Psi$  therefore sets the grid from the reference distribution. The analysis in Section V-B2 showed that LVLM measurements of the same physical quantity vary across images by an amount that depends on the constraint. Let  $r_k$  denote the measurement of the target property from reference  $k$ , and let  $\rho = \max_k r_k - \min_k r_k$  denote the spread across the  $K$  references. **CASCADE++** finds the narrowest cell of width  $w$  and offset  $a$  that holds every  $r_k$  with at least a margin  $m$  from each boundary, judging a query normal when its reading lands in this cell and a violation otherwise. Two backbone-dependent constants set the resolution: the margin  $m$ , the smallest separation the LVLM can reliably resolve between a reading and a cell boundary, and the minimum width  $w_{\min}$ , the narrowest cell at which it can still tell adjacent cells apart. Both are calibrated once per backbone on synthetic probes and held fixed across constraints ( $m=25$  px and  $w_{\min}=60$  px for

TABLE II: Calibration rule ablation on Balanced Accuracy (mean  $\pm$  std over five random reference selections). Grid parameterization per strategy: **Fixed** ( $w=100$ ,  $a^*$ ), **Dyn-size** ( $w=\text{tgt}/2$ ,  $a^*$ ), **Dyn-2 $\times$**  ( $w=\text{tgt}/4$ ,  $a^*$ ),  $a_{\text{adv}}$  ( $w^*$ , adversarial  $a$ ), and **CASCADE++** ( $w^*$ ,  $a^*$ ).

| Constraint | Fixed           | Dyn-size        | Dyn-2 $\times$  | $a_{\text{adv}}$ | <b>CASCADE++</b>               |
|------------|-----------------|-----------------|-----------------|------------------|--------------------------------|
| BB-Cereal  | 77.6 $\pm$ 9.9  | 71.4 $\pm$ 6.7  | 78.6 $\pm$ 10.1 | 63.8 $\pm$ 8.6   | <b>82.9<math>\pm</math>4.0</b> |
| JB-Fruit   | 90.0 $\pm$ 11.3 | 95.9 $\pm$ 1.9  | 76.9 $\pm$ 6.1  | 85.7 $\pm$ 9.5   | <b>94.4<math>\pm</math>3.9</b> |
| JB-Text    | 83.3 $\pm$ 6.6  | 81.4 $\pm$ 10.6 | 82.6 $\pm$ 2.3  | 84.3 $\pm$ 10.9  | <b>96.9<math>\pm</math>0.1</b> |
| JB-Liquid  | 97.8 $\pm$ 1.0  | 25.6 $\pm$ 7.9  | 72.1 $\pm$ 0.9  | 84.0 $\pm$ 4.3   | <b>95.9<math>\pm</math>0.2</b> |
| SB-Length  | 92.2 $\pm$ 0.4  | 50.0 $\pm$ 2.7  | 96.4 $\pm$ 0.5  | 86.7 $\pm$ 2.0   | <b>92.4<math>\pm</math>4.9</b> |
| <b>Avg</b> | <b>88.2</b>     | <b>64.9</b>     | <b>81.3</b>     | <b>80.9</b>      | <b>92.5</b>                    |

GPT-5-image-mini, the smallest values reaching 95% probe accuracy).

Concretely, the cell  $[a, a+w]$  must satisfy

$$\begin{aligned} a + m &\leq r_k \leq a + w - m, \quad \forall k \in \{1, \dots, K\}, \\ w &\geq w_{\min}, \end{aligned} \quad (3)$$

so that every reference reading sits at least a margin  $m$  from either boundary. Eliminating  $a$ , the tightest feasible width is

$$w^* = \max (w_{\min}, \rho + 2m), \quad (4)$$

and an offset that places every reference at least  $m$  from either boundary is

$$a^* = \min_k r_k - m. \quad (5)$$

Together,  $w^*$  and  $a^*$  constitute the configuration  $s_i$  that  $\Psi$  returns for a calibration constraint. Since  $m$  and  $w_{\min}$  are fixed per backbone and only the references enter  $\rho$ , no test-time reading touches the calibration. When a constraint compares several groups or spans two axes, the same containment conditions are solved jointly over a shared cell width. The calibration is a one-time cost per backbone: the constants reflect the backbone’s measurement noise, not any dataset or category, and are reused unchanged from MVTec-LOCO to VID-AD (Section VIII-C).

Both extensions follow one principle: the constraint type fixes the overlay, while the constraint’s logical structure and the reference distribution fix its configuration.

## VIII. EXPERIMENTS

### A. Experimental Setup

We evaluate **CASCADE** on two benchmarks. MVTec-LOCO [1] provides five product categories, Breakfast Box (BB), Juice Bottle (JB), Pushpins (PP), Screw Bag (SB), and Splicing Connectors (SC), comprising 16 logical constraints, 10 binding and 6 calibration; a single category may contain both types. Per-constraint analyses name a constraint by its category and measured target (e.g., JB-Liquid), and the measurement labels *Count*, *Ratio*, *Length*, and *Fill* denote PP-Count, BB-Cereal, SB-Length, and JB-Liquid. VID-AD [25] spans ten scenarios under five capture conditions (white, cable, and mesh background, low-light, and blurry); its category specifications

yield 21 constraints, 17 binding and 4 calibration, so both scaffold types are exercised on this benchmark as well.

For MVTec-LOCO we adopt the natural-language constraints from LogicQA [4] so that all vision-language methods share the same constraint-input regime; for VID-AD we derive them from the dataset authors’ category specifications [25]. The baselines were originally reported on GPT-4o; to isolate scaffold design from backbone choice we reproduce the publicly available ones, LogicAD [5] and LogicQA [4], on GPT-5-image-mini with their own inference procedures and reference budgets intact. LogSAD [17] routes the task through specialized foundation models rather than a single swappable chat LVLm, so we run the authors’ released code under our  $K=3$  protocol instead of porting it.

**Implementation details.** The primary backbone is GPT-5-image-mini at high reasoning effort, used identically for **CASCADE** and the reproduced baselines. Object masks come from the official SAM3 checkpoint via promptable concept segmentation on the constraint’s target objects, computed once per image and shared across its constraints.  $\Psi$  runs once per category, so the only per-query overhead is the single SAM3 segmentation pass (0.59 s, under 4 GiB of VRAM on one RTX A5000). All few-shot results draw  $K=3$  references uniformly at random with fixed seeds; constraint-level ablations (Tables I–II) average over five such selections, where single-verdict variance is largest, and image-level results (Tables III–IV) over three, since a full-system pass is an order of magnitude more expensive. Verification issues one LVLm call per constraint, against LogicQA’s five, so the accuracy reported below does not come at the cost of inference budget. All prompt templates, the constraint parser, SAM3 settings, decoding parameters, the calibration-probe generators, the grid solver, and the reference-set seeds are released in our repository, enabling exact reproduction of every table.

We test generalization across benchmarks and backbones (Section VIII-C), ablate the decomposition into its two scaffolds (Section VIII-D), and examine how performance depends on the number of references (Section VIII-E).

### B. Main Results — MVTec-LOCO

With three references **CASCADE++** reaches  $93.2 \pm 0.8$  AUROC over three reference sets without training (Table III). Under the same backbone and reference budget it improves over the strongest reproduced baseline (LogicQA<sup>†</sup>, 86.2) by 7.0, and it exceeds the full-shot state of the art (LogSAD, 89.3) by 3.9. LogSAD reports 89.3 with the full training set; under our  $K=3$  protocol the authors’ released code yields 84.6.

Base **CASCADE** reaches only 81.4, below both reproduced baselines. This is the image-level counterpart of what Table I shows at the constraint level: a scaffold whose configuration misaligns with the constraint misleads the verifier more than no scaffold at all. Deriving the configuration from the constraint and the references adds +11.8 and reverses the ordering. The gain concentrates on calibration-heavy categories, where the reference-calibrated grid replaces the fixed default: Juice Bottle +21.0, Screw Bag +20.3, and Splicing Connector +13.9.

**CASCADE++** is also the only framework that stays above 87 on every category, whereas every baseline collapses on at least one—PatchCore and EfficientAD near 57 on Screw Bag, LogicAD 73.4 on Splicing Connector, LogicQA 71.5 on Screw Bag—since routing each constraint to its matched scaffold keeps one failure mode from dominating the score.

The gain comes from the scaffold, not the backbone. Reproducing LogicAD and LogicQA on GPT-5-image-mini yields 85.5 and 86.2, barely above their GPT-4o results (86.0 and 87.6): a stronger backbone reasons better but cannot recover evidence the raw image never made visible. The same scaffold transfers across backbones, holding 92.7 on Gemini-3.1-Flash-Lite and 90.6 on Qwen3-VL-32B, and even the 8B model (83.2) approaches the full-shot vision state of the art.

### C. Main Results — VID-AD

We verify generalization by applying the identical pipeline, without any parameter adjustment, to VID-AD (Table IV). **CASCADE++** achieves  $94.5 \pm 2.1$ , exceeding the strongest baseline (83.1) by 11.4 points while using only three references against the full training set the other methods require. The gain is consistent across all five capture conditions, including low-light (93.0) and blurry (96.2), where feature-based methods degrade most: EfficientAD falls to 47.9 and 48.0 respectively. Because the scaffold makes the constraint-relevant evidence explicit before the verifier reads it, the capture condition affects the verdict far less than it affects methods that must recover that evidence from raw features. The conditions also probe how far the pipeline tolerates degraded segmentation, since SAM3’s mask quality drops under low light and blur. Performance holds because the scaffolds require only approximate masks: the disambiguation overlay stays valid as long as instances remain *separated*, and the alignment grid anchors to a rotated box that absorbs boundary noise. What the scaffolds cannot absorb is a wrong instance partition, since a merged mask misleads the overlay in the same way a misaligned ID does (Table I). Segmentation is thus the remaining perceptual bottleneck of the pipeline.

### D. Decomposition Ablation

Table V independently enables or disables each scaffold. The disambiguation scaffold raises the binding score from 72.6 to 80.9 (+8.3) while calibration is essentially unchanged ( $62.7 \rightarrow 59.9$ ). Conversely, the alignment scaffold raises calibration from 62.7 to 70.0 (+7.3) while binding remains unchanged ( $72.6 \rightarrow 72.9$ ). This double dissociation confirms that the two failure modes are independent and licenses routing each constraint to a single matched scaffold. Combining both raises overall AUROC from 74.5 to 93.2.

### E. Reference Sensitivity

The calibration grid estimates its cell width  $w^*$  from  $K$  references, so  $K$  governs how reliably the normal range is recovered. The disambiguation scaffold derives its configuration from the constraint text and is unaffected by  $K$ , so this analysis focuses on calibration constraints.

TABLE III: Image-level AUROC (%) on the MVTec-LOCO logical subset. The first block is trained on the full normal set. In the second block, rows use  $K=3$  references except LogicAD, which uses one by design, and the LogSAD row marked full-shot; dashes are per-category scores the LogSAD paper does not report. Rows without a marker are quoted from the original papers, and  $^\dagger$  marks results we produced, keeping each method’s own inference procedure and reference budget. Bold marks the best result among methods on GPT-5-image-mini. All entries are single numbers; variance across reference sets for **CASCADE++** is reported in Section VIII-B and analyzed in Section VIII-E.

| Method                                | Backbone              | Avg         | BB          | JB          | PP          | SB          | SC          |
|---------------------------------------|-----------------------|-------------|-------------|-------------|-------------|-------------|-------------|
| <i>Vision (full-shot)</i>             |                       |             |             |             |             |             |             |
| PatchCore [2]                         | CNN                   | 74.0        | 74.8        | 93.9        | 63.6        | 57.8        | 79.2        |
| GCAD [1]                              | CNN                   | 86.0        | 87.0        | 100.0       | 97.5        | 56.0        | 89.7        |
| EfficientAD [3]                       | CNN                   | 86.8        | 85.5        | 98.4        | 97.7        | 56.7        | 95.5        |
| AST [26]                              | CNN                   | 79.7        | 80.0        | 91.6        | 65.1        | <b>80.1</b> | 81.8        |
| <i>Vision-language</i>                |                       |             |             |             |             |             |             |
| WinCLIP [27]                          | CLIP                  | 64.3        | 57.6        | 75.1        | 54.9        | 69.5        | 64.5        |
| LogicAD [5]                           | GPT-4o                | 86.0        | 93.1        | 81.6        | 98.1        | 83.8        | 73.4        |
| LogicQA [4]                           | GPT-4o                | 87.6        | 87.6        | 88.2        | 98.4        | 71.5        | 92.4        |
| LogSAD [17] (full-shot)               | GPT-4o                | <b>89.3</b> | —           | —           | —           | —           | —           |
| LogSAD $^\dagger$ [17]                | GPT-4o                | 84.6        | 90.9        | 85.4        | 81.4        | 81.7        | 83.7        |
| LogicAD $^\dagger$                    | GPT-5-image-mini      | 85.5        | <b>94.7</b> | 80.5        | <b>97.6</b> | 79.3        | 75.3        |
| LogicQA $^\dagger$                    | GPT-5-image-mini      | 86.2        | 91.6        | 88.7        | 97.3        | 68.0        | 85.3        |
| <b>CASCADE<math>^\dagger</math></b>   | GPT-5-image-mini      | 81.4        | 90.0        | 73.7        | 93.8        | 75.5        | 73.8        |
| <b>CASCADE++<math>^\dagger</math></b> | GPT-5-image-mini      | <b>93.2</b> | 91.4        | <b>94.7</b> | 96.6        | <b>95.8</b> | <b>87.7</b> |
| <b>CASCADE++<math>^\dagger</math></b> | Gemini-3.1-Flash-Lite | 92.7        | 89.9        | 92.9        | 96.3        | 95.2        | 89.2        |
| <b>CASCADE++<math>^\dagger</math></b> | Qwen3-VL-32B          | 90.6        | 91.4        | 84.5        | 100.0       | 91.3        | 85.6        |
| <b>CASCADE++<math>^\dagger</math></b> | Qwen3-VL-8B           | 83.2        | 87.7        | 75.7        | 82.1        | 86.0        | 84.4        |

TABLE IV: Main results on VID-AD (image-level AUROC, %). W, C, M, L, and B denote white, cable, mesh background, low-light, and blurry conditions, respectively. Full-shot methods use the complete training set, whereas **CASCADE++** uses  $K=3$  references with no training.

| Method                  | W           | C           | M           | L           | B           | Mean $\pm$ Std                 |
|-------------------------|-------------|-------------|-------------|-------------|-------------|--------------------------------|
| <i>Full-shot</i>        |             |             |             |             |             |                                |
| PatchCore [2]           | 40.4        | 45.0        | 48.0        | 46.9        | 42.0        | 44.5 $\pm$ 2.9                 |
| EfficientAD [3]         | 52.6        | 60.2        | 60.8        | 47.9        | 48.0        | 53.9 $\pm$ 5.7                 |
| PaDiM [28]              | 34.8        | 35.4        | 42.8        | 35.4        | 40.3        | 37.8 $\pm$ 3.2                 |
| VAE [29]                | 44.8        | 49.4        | 45.4        | 47.2        | 39.3        | 45.2 $\pm$ 3.4                 |
| AnoGAN [30]             | 48.9        | 54.8        | 49.2        | 47.4        | 46.8        | 49.4 $\pm$ 2.8                 |
| CSAD [12]               | 69.3        | 62.4        | 64.1        | 69.4        | 65.7        | 66.2 $\pm$ 2.8                 |
| VID-AD baseline [25]    | 82.5        | 81.1        | 84.8        | 84.2        | 82.6        | 83.1 $\pm$ 1.3                 |
| <i>Training-free</i>    |             |             |             |             |             |                                |
| UniVAD [15]             | 57.4        | 59.2        | 57.0        | 57.3        | 55.9        | 57.4 $\pm$ 1.1                 |
| <b>CASCADE++ (Ours)</b> | <b>94.7</b> | <b>91.4</b> | <b>97.1</b> | <b>93.0</b> | <b>96.2</b> | <b>94.5<math>\pm</math>2.1</b> |

At  $K=1$  a single reference cannot span the normal range, causing  $w^*$  to be underestimated. The resulting grid is overly narrow and misclassifies normal queries as violations. JB-Fruit, for example, reaches only 70.7 balanced accuracy with a standard deviation of 22.5 across reference sets, and BB-Cereal only 71.8 $\pm$ 12.6. As  $K$  increases to 3, the estimated  $w^*$  converges toward its full-reference value and the standard deviation contracts sharply. Accuracy follows the same trend, with JB-Fruit rising to 94.4 $\pm$ 3.9 and BB-Cereal to 82.9 $\pm$ 4.0. The margin  $m$  in the calibration objective absorbs the residual from estimating the range with finitely many references, which is why the parameter convergence translates into stable accuracy. We therefore fix  $K=3$ , which balances reference acquisition

TABLE V: Decomposition ablation on MVTec-LOCO (image-level AUROC, %). All rows use the **CASCADE++** configuration rules and differ only in which scaffolds are enabled; “No scaffold” verifies the same checklist on the raw query image. The Binding and Calibration columns score each image by the violated fraction of binding-only and calibration-only sub-constraints, and so miss anomalies of the other type; Overall uses all sub-constraints, which is why it exceeds either column alone.

| Configuration           | Disambig. | Alignment | Binding     | Calibration | Overall     |
|-------------------------|-----------|-----------|-------------|-------------|-------------|
| No scaffold             | —         | —         | 72.6        | 62.7        | 74.5        |
| Disambiguation only     | ✓         | —         | 80.9        | 59.9        | 81.8        |
| Alignment only          | —         | ✓         | 72.9        | 70.0        | 79.5        |
| <b>CASCADE++ (both)</b> | ✓         | ✓         | <b>83.8</b> | <b>74.5</b> | <b>93.2</b> |

cost against detection accuracy.

## IX. CONCLUSION

In this paper, we show that the limitation of checklist-based LVLM verification for LAD is perceptual rather than textual, manifesting as instance conflation for binding constraints and scale drift for calibration constraints. Based on this finding, we propose **CASCADE**, a training-free framework that extends checklist decomposition to the image through constraint-specific visual scaffolds, and **CASCADE++**, which derives each scaffold’s configuration from the constraint structure and the reference distribution. Experiments across two benchmarks and multiple backbones confirm that constraint-specific visual adaptation consistently improves LAD performance, reaching 93.2 AUROC on MVTec-LOCO and 94.5 on VID-AD without any training. Two dependencies bound the current framework.

First, the constraints are inputs: they are specified once per category by a domain expert, and **CASCADE++** verifies them but does not discover, repair, or update them, so handling incomplete, ambiguous, or evolving constraint sets remains open. Second, both scaffolds inherit the quality of the underlying segmentation; boundary noise is absorbed by design, but a merged or split mask misleads the overlay just as a misaligned configuration does (Section VIII-C). Future work may extend the constraint typology beyond binding and calibration, derive scaffolds for constraints that combine both, and couple the scaffold with segmentation confidence so that an unreliable mask is flagged before it reaches the verifier.

#### ACKNOWLEDGMENT

This work was partly supported by Institute of Information & communications Technology Planning & Evaluation (IITP) under the Artificial Intelligence Convergence Innovation Human Resources Development (IITP-2026-RS-2023-00255968) grant funded by the Korea government (MSIT), IITP grant funded by the Korea government (MSIT) (No. RS-2022-II220680, Abductive inference framework using omni-data for understanding complex causal relations; RS-2026-25617480), the Nano & Material Technology Development Program through the National Research Foundation of Korea (NRF) funded by Ministry of Science and ICT (RS-2024-00444182), and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2026-25518808).

#### REFERENCES

- [1] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "Beyond dents and scratches: Logical constraints in unsupervised anomaly detection and localization," *International Journal of Computer Vision*, vol. 130, no. 4, pp. 947–969, 2022.
- [2] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 14 298–14 308.
- [3] K. Batzner, L. Heckler, and R. König, "Efficientad: Accurate visual anomaly detection at millisecond-level latencies," in *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, 2024, pp. 128–138.
- [4] Y. Kwon, D. Moon, Y. Oh, and H. Yoon, "Logicqa: Logical anomaly detection with vision language model generated questions," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, 2025, pp. 411–432.
- [5] E. Jin, Q. Feng, Y. Mou, G. Lakemeyer, S. Decker, O. Simons, and J. Stegmaier, "Logicad: Explainable anomaly detection via vlm-based text feature extraction," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 4, 2025, pp. 4129–4137.
- [6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.
- [7] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in neural information processing systems*, vol. 36, pp. 46 534–46 594, 2023.
- [8] A. Shtedritski, C. Rupprecht, and A. Vedaldi, "What does clip know about a red circle? visual prompt engineering for vlms," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2023, pp. 11 953–11 963.
- [9] X. Lei, Z. Yang, X. Chen, P. Li, and Y. Liu, "Scaffolding coordinates to promote vision-language coordination in large multi-modal models," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 2886–2903.
- [10] A. Izadi, M. Banayeezanade, F. Askari, A. Rahimiakbar, M. Vahedi, H. Hasani, and M. Baghshah, "Visual structures help visual reasoning: Addressing the binding problem in LVLMS," *Advances in Neural Information Processing Systems*, vol. 38, pp. 162 930–162 968, 2025.
- [11] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, "Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v," *arXiv preprint arXiv:2310.11441*, 2023.
- [12] Y.-H. Hsieh and S.-H. Lai, "Csad: Unsupervised component segmentation for logical anomaly detection," in *35th British Machine Vision Conference (BMVC)*. BMVA, 2024.
- [13] M. Fučka, V. Zavrtnik, and D. Skočaj, "Salad-semantics-aware logical anomaly detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 21 843–21 852.
- [14] Y. Peng, X. Lin, N. Ma, J. Du, C. Liu, C. Liu, and Q. Chen, "Sam-lad: Segment anything model meets zero-shot logic anomaly detection," *Knowledge-Based Systems*, vol. 314, p. 113176, 2025.
- [15] Z. Gu, B. Zhu, G. Zhu, Y. Chen, M. Tang, and J. Wang, "Univad: A training-free unified model for few-shot visual anomaly detection," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 15 194–15 203.
- [16] Y. Zhang, Y. Cao, X. Xu, and W. Shen, "Logiccode: an llm-driven framework for logical anomaly detection," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 7712–7723, 2025.
- [17] J. Zhang, G. Wang, Y. Jin, and D. Huang, "Towards training-free anomaly detection with vision and language foundation models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 15 204–15 213.
- [18] Y. Hu, W. Shi, X. Fu, D. Roth, M. Ostendorf, L. Zettlemoyer, N. A. Smith, and R. Krishna, "Visual sketchpad: Sketching as a visual chain of thought for multimodal language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 139 348–139 379.
- [19] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, A. Ananthram *et al.*, "Openai gpt-5 system card," *arXiv preprint arXiv:2601.03267*, 2025.
- [20] K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann, "The balanced accuracy and its posterior distribution," in *2010 20th International Conference on Pattern Recognition*. IEEE, 2010, pp. 3121–3124.
- [21] S. Tong, Z. Liu, Y. Zhai, Y. Ma, Y. LeCun, and S. Xie, "Eyes wide shut? exploring the visual shortcomings of multimodal llms," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 9568–9578.
- [22] Google DeepMind, "Gemini 3.1 flash-lite model card," <https://deepmind.google/models/model-cards/gemini-3-1-flash-lite/>, Mar. 2026, accessed: 2026-06-07.
- [23] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge *et al.*, "Qwen3-vl technical report," *arXiv preprint arXiv:2511.21631*, 2025.
- [24] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang *et al.*, "Sam 3: Segment anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.
- [25] H. Nakata, Y. Zou, S. Sakai, S. Maeda, C. Gu, Y. Wei, S. Gao, and C. Zhang, "VID-AD: A dataset for image-level logical anomaly detection under vision-induced distraction," *arXiv preprint arXiv:2603.13964*, 2026.
- [26] M. Rudolph, T. Wehrbein, B. Rosenhahn, and B. Wandt, "Asymmetric student-teacher networks for industrial anomaly detection," in *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, 2023, pp. 2592–2602.
- [27] J. Jeong, Y. Zou, T. Kim, D. Zhang, A. Ravichandran, and O. Dabeer, "Winclip: Zero-/few-shot anomaly classification and segmentation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 19 606–19 616.
- [28] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "Padim: a patch distribution modeling framework for anomaly detection and localization," in *International conference on pattern recognition*. Springer, 2021, pp. 475–489.
- [29] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proceedings of the 2nd International Conference on Learning Representations*, 2014.
- [30] T. Schlegl, P. Seeböck, S. M. Waldstein, U. Schmidt-Erfurth, and G. Langs, "Unsupervised anomaly detection with generative adversarial networks to guide marker discovery," in *Information Processing in Medical Imaging*, 2017, pp. 146–157.